

TEMPLATE

Prof. André Backes

Template em C++

- Recurso que permite escrever código genérico em C++
 - Ao invés de escrever múltiplas versões de uma mesma função ou classe para diferentes tipos de dados (como int, float, double, etc), é possível criar um único modelo que funciona com qualquer tipo

Template em C++

- Especialmente útil quando se deseja aplicar o mesmo algoritmo ou lógica a diferentes tipos de dados
 - Permite criar um único modelo que funciona com qualquer tipo
 - Promove a reutilização, a manutenibilidade e a eficiência do código, já que um único template pode substituir diversas implementações específicas
 - Amplamente usados na **STL** - *Standard Template Library*

Template em C++

- Existem dois tipos principais de template em C++:
 - Template de Função
 - Permite criar funções genéricas que operam sobre diferentes tipos de dados.
 - Template de Classe
 - Permite criar classes genéricas que podem armazenar qualquer tipo de elemento.

TEMPLATE DE FUNÇÃO

Template de Função

- Queremos criar uma função para retornar o maior dentre dois números
 - A implementação depende do tipo dos dados que estão sendo comparados
- Sobrecarga de função permite repetir nomes de funções
 - Mas requer implementações separadas

```
int maior(int x, int y){  
    return (x > y) ? x : y;  
}  
  
double maior(double x, double y){  
    return (x > y) ? x : y;  
}
```

Template de Função

- Além do nome, o código da função também está repetido
 - Nesse exemplo, a comparação de valores não depende dos tipos comparados

```
int maior(int x, int y){  
    return (x > y) ? x : y;  
}  
  
double maior(double x, double y){  
    return (x > y) ? x : y;  
}
```

Template de Função

- Um template permite reutilizar o código de uma função para operar com diferentes tipos, sem precisar escrever implementações separadas para cada tipo
 - É modelo genérico que permite definir uma função sem especificar explicitamente os tipos de dados que ela manipula
 - O compilador gera versões específicas da função com base nos tipos utilizados durante a chamada

Criando e usando um template de função

- A sintaxe de um template de função segue a forma geral ao lado
 - T é um parâmetro de tipo genérico
 - Substitui os tipos que são possíveis para essa função

```
template <typename T>  
T nome_função(T a, T b) {  
    // código da função  
}
```

Criando e usando um template de função

- Criando a função maior() com um template
 - A função é usada para os tipos **int** e **double** com o mesmo template

```
template <typename T>
T maior(T x, T y) {
    return (x > y) ? x : y;
}

int main(){
    // instanciação explícita
    cout << maior<int>(12,7) << endl;
    cout << maior<double>(10,2.5) << endl;

    // instanciação implícita
    cout << maior(4,9) << endl;
    cout << maior(10.0,2.5) << endl;

    return 0;
}
```

Instanciação do template

- Ocorre na primeira chamada da função para um tipo específico
 - Apenas quando a função é chamada é que o compilador substituirá T pelo tipo utilizado
 - O compilador gera, automaticamente, aquela versão da função

```
template <typename T>
T maior(T x, T y) {
    return (x > y) ? x : y;
}

int main(){
    // instanciação explícita
    cout << maior<int>(12,7) << endl;
    cout << maior<double>(10,2.5) << endl;

    // instanciação implícita
    cout << maior(4,9) << endl;
    cout << maior(10.0,2.5) << endl;

    return 0;
}
```

Instanciação do template

- Instanciação pode ser
 - Explícita, o tipo é especificado entre <>
 - Implícita, o compilador tenta descobrir o tipo automaticamente

```
template <typename T>
T maior(T x, T y) {
    return (x > y) ? x : y;
}

int main(){
    // instanciação explícita
    cout << maior<int>(12,7) << endl;
    cout << maior<double>(10,2.5) << endl;

    // instanciação implícita
    cout << maior(4,9) << endl;
    cout << maior(10.0,2.5) << endl;

    return 0;
}
```

Instanciação do template

- A instanciação da função para um tipo ocorre uma única vez
 - Quando a função é chamada a primeira vez para o tipo **int** ocorre a instanciação
 - O compilador vê que existe apenas o template da função, mas não a sua implementação
 - Quando a função é novamente chamada para o tipo **int** não há necessidade de criar novamente essa função, pois ela já existe.
 - O mesmo acontece quando a função é chamada a primeira vez para o tipo **double**.

```
template <typename T>
T maior(T x, T y) {
    return (x > y) ? x : y;
}

int main(){
    // instanciação explícita
    cout << maior<int>(12,7) << endl;
    cout << maior<double>(10,2.5) << endl;

    // instanciação implícita
    cout << maior(4,9) << endl;
    cout << maior(10.0,2.5) << endl;

    return 0;
}
```

Parâmetros de tipos diferentes

- Podemos definir um parâmetro de um tipo pré-determinado
 - Terá o mesmo tipo entre as várias chamadas da função enquanto **T** pode assumir qualquer tipo
 - Neste exemplo, a função é instanciada duas vezes, na primeira com o tipo `int` e na segunda com o tipo `double`.
 - Na primeira chamada o resultado da função é truncado, pois o tipo de retorno da função segue o tipo do primeiro parâmetro, `int`

```
template <typename T>
T soma(T x, double y) {
    return x + y;
}

int main() {
    cout << soma<int>(1, 2.5) << endl;
    cout << soma<double>(10.0, 2.5) << endl;

    return 0;
}
```

Parâmetros de tipos diferentes

- Se o parâmetro do template é de um tipo pré-determinado, então também podemos definir um valor padrão

```
template <typename T>
T soma(T x, double y = 2.5) {
    return x + y;
}

int main() {
    cout << soma<int>(1) << endl;
    cout << soma<double>(10.0) << endl;
    cout << soma<double>(10.0, 4.5) << endl;

    return 0;
}
```

Múltiplos tipos no Template

- Podemos definir um template que aceita mais de um tipo
 - Basta declarar vários parâmetros de tipo
 - O template aceita dois parâmetros de tipos diferentes, T1 e T2
 - Maior flexibilidade no uso do template
 - Evita conversões de tipos desnecessárias ou sem sentido.

```
template <typename T1, typename T2>
void exhibir(T1 a, T2 b) {
    cout << "Valor 1: " << a << ", Valor 2: " << b << endl;
}

int main(){
    exhibir(10, 5.5);
    exhibir("Texto", 42);

    return 0;
}
```


Template usando **auto** (padrão C++20)

- Podemos criar um template de função usando a palavra chave **auto** no retorno e nos tipos dos parâmetros
 - Isso informa ao compilador que o tipo do parâmetro será o mesmo do valor da chamada da função
 - Isso permite criar templates com vários tipos

```
// emplace de função abreviada usando auto
auto maior(auto x, auto y) {
    return (x > y) ? x : y;
}
```

```
// Código template equivalente
template <typename T1, typename T2>
auto maior(T1 x, T2 y) {
    return (x > y) ? x : y;
}
```

Sobrecarga de template

- Podemos fazer a sobrecarga de um template assim como a de uma função
 - O compilador fica responsável por selecionar qual template usar
- Nesse exemplo, três funções diferentes são instanciadas
 - A primeira chamada segue o primeiro template: os atributos são do mesmo tipo (int)
 - As duas chamadas seguintes utilizam o segundo template, pois possuem tipos, e a ordem deles, diferentes

```
template <typename T>
auto soma(T x, T y) {
    return x + y;
}

template <typename T1, typename T2>
auto soma(T1 x, T2 y) {
    return x + y;
}

int main() {
    // instancia soma<int>()
    cout << soma(1,2) << endl;
    // instancia soma<int,double>()
    cout << soma(1,2.5) << endl;
    // instancia soma<double,int>()
    cout << soma(1.5,2.5) << endl;

    return 0;
}
```

Especialização

- Consiste em fornecer uma versão alternativa da função template quando ela é utilizada com um tipo específico
 - Sintaxe similar à de uma função comum
 - Indicação de template<> e a definição da função com o tipo desejado
- Funciona como a sobrecarga de função
 - Permite modificar a lógica da função para tipos específicos

```
// Template genérico
template <typename T>
void imprimir(T valor) {
    cout << "Valor generico: " << valor << endl;
}

// Especialização para string
template <>
void imprimir<string>(string valor) {
    cout << "Texto: " << valor << endl;
}

int main() {
    imprimir(42);
    imprimir(3.14);
    imprimir("Oi");
    return 0;
}
```

Especialização

- Não há necessidade usar especialização quando uma simples sobrecarga de função pode resolver o problema
 - Analisar caso a caso

```
// Template genérico
template <typename T>
void imprimir(T valor) {
    cout << "Valor generico: " << valor << endl;
}

// sobrecarga da função para string
void imprimir(string valor) {
    cout << "Texto: " << valor << endl;
}

int main() {
    imprimir(42);
    imprimir(3.14);
    imprimir("Oi");
    return 0;
}
```

TEMPLATE DE CLASSE

Template de Classe

- Precisamos criar uma classe que represente uma pilha estática
 - Implementação para int

```
class Pilha {
private:
    int topo, tamanho, *elementos;

public:
    Pilha(int cap) : topo(0), tamanho(cap) {
        elementos = new int[tamanho];
    }
    ~Pilha() {
        delete[] elementos;
    }

    void insere(int elemento) {
        if(topo == tamanho)
            cout << "Pilha cheia!" << endl ;
        else
            elementos[topo++] = elemento;
    }

    void remove(){
        if(topo == 0)
            cout << "Pilha vazia!" << endl ;
        else
            topo--;
    }

    void imprime() {
        if(topo == 0) cout << "Pilha vazia." << endl;
        else{
            cout << "Elementos da Pilha: " << endl;
            for(int i = 0; i < topo; ++i)
                cout << elementos[i] << endl;
        }
    }
};
```

Template de Classe

- E se precisarmos armazenar outro tipo?
 - Necessário criar uma nova versão da classe
 - Alterar apenas alguns pontos chaves
 - a declaração do ponteiro elementos
 - a alocação desse ponteiro dentro do construtor
 - o tipo do parâmetro passado para a função insere()
 - Grande quantidade de código duplicado

Template de Classe

- Um template permite reutilizar o código da classe para diferentes tipos
 - Sem precisar escrever implementações separadas para cada tipo
- Sintaxe de um template de classe
 - T é um parâmetro de tipo genérico
 - Substitui os tipos que são possíveis para os atributos e parâmetros dos métodos da classe

```
template <typename T>
class nome_classe {
    private:
        T var;
        ...
    public:
        T nome_função(T arg);
        ...
};
```


Template de Classe

- Adaptando a classe Pilha para trabalhar com um template de classe
 - Não é necessário alterar outros métodos e atributos
 - A lógica de como funciona uma pilha não foi alterada, apenas o tipo armazenado
 - Podemos definir atributos com tipos fixos
 - topo, tamanho, ...

```
template <typename T>
class Pilha {
private:
    int topo, tamanho;
    T* elementos;

public:
    Pilha(int cap) : topo(0), tamanho(cap) {
        elementos = new T[tamanho];
    }

    ~Pilha() {
        delete[] elementos;
    }

    void insere(T elemento) {
        if(topo == tamanho)
            cout << "Pilha cheia!" << endl ;
        else
            elementos[topo++] = elemento;
    }

    void remove(){
        if(topo == 0) cout << "Pilha vazia!" << endl ;
        else topo--;
    }

    void imprime() {
        if(topo == 0) cout << "Pilha vazia." << endl;
        else{
            cout << "Elementos da Pilha: " << endl;
            for(int i = 0; i < topo; ++i)
                cout << elementos[i] << endl;
        }
    }
};
```

Instanciação do template

- Ocorre quando um objeto da classe é criado para um tipo específico
 - Substitui T pelo tipo utilizado na criação do objeto
 - O compilador gera, automaticamente, aquela versão da classe

```
int main() {  
    Pilha<int> p1(10);  
    p1.insere(5);  
    p1.insere(15);  
    p1.imprime();  
  
    Pilha<double> p2(10);  
    p2.insere(10.3);  
    p2.insere(20.2);  
    p2.imprime();  
  
    return 0;  
}
```

Instanciação do template

- A instanciação da classe é sempre explícita
 - Na instanciação do objeto não está definido o tipo de dado que será armazenado na pilha, apenas o tamanho dela

```
int main() {  
    Pilha<int> p1(10);  
    p1.insere(5);  
    p1.insere(15);  
    p1.imprime();  
  
    Pilha<double> p2(10);  
    p2.insere(10.3);  
    p2.insere(20.2);  
    p2.imprime();  
  
    return 0;  
}
```

Parâmetros de tipos diferentes

- Podemos definir um template que aceita mais de um tipo
 - Basta declarar vários parâmetros de tipo
 - Maior flexibilidade no uso do template
 - Aqui fazemos uso da classe stringstream, que é uma maneira conveniente para converter entre strings e outros tipos numéricos

```
template <typename TipoEntrada, typename TipoSaida>
class Conversor {
private:
    TipoEntrada valorOriginal;
    TipoSaida valorConvertido;
public:
    Conversor(TipoEntrada valor) : valorOriginal(valor) {
        stringstream ss;
        ss << valorOriginal;
        ss >> valorConvertido;
    }

    void exibirConversao() const {
        cout << "Original: " << valorOriginal
              << " | Convertido: " << valorConvertido << "\n";
    }
};

int main() {
    // Conversão de int para string
    Conversor<int, string> c1(123);
    c1.exibirConversao();

    // Conversão de double para int
    Conversor<double, int> c2(3.14159);
    c2.exibirConversao();

    // Conversão de string para int
    Conversor<string, int> c3("456");
    c3.exibirConversao();

    return 0;
}
```

Funções membros

- Implementação fora da classe?
 - Necessário definir o protótipo do método dentro da classe
 - Para que não seja confundido com uma função externa
 - Colocar o identificador da classe e o operador de resolução de escopo antes da função
 - Necessário incluir uma nova declaração de parâmetro de template

```
template <typename T>
class Coordenada{
private:
    T m_x, m_y;

public:
    Coordenada(T x, T y): m_x{x}, m_y{y}{
    }
    bool ehIgual(const Coordenada<T>& c);
};

// Função membro definida fora da classe exige
// nova declaração de parâmetro de template
template <typename T>
bool Coordenada<T>::ehIgual(const Coordenada<T>& c){
    return m_x == c.m_x && m_y == c.m_y;
}

int main() {
    Coordenada<int> c1(10, 25);
    Coordenada<int> c2(14, 32);

    if(c1.ehIgual(c2))
        cout << "iguais" << endl;
    else
        cout << "diferentes" << endl;

    return 0;
}
```

Template de funções e sobrecarga de operador

- Template da função maior() funciona muito bem com os tipos **int** e **double**
 - Porém, o compilador irá gerar um erro na chamada da função se tentarmos usá-lo com um objeto da classe Pessoa

```
class Pessoa {
private:
    string nome;
    int idade;

public:
    Pessoa(string n, int i): nome{n}, idade{i}{

    void exibir(){
        cout << "Nome: " << nome << ", Idade: " << idade << endl;
    }
};

template <typename T>
T maior(T x, T y) {
    return (x > y) ? x : y;
}

int main() {
    Pessoa p1("Maria", 25);
    Pessoa p2("Joao", 32);

    Pessoa p3(maior(p1,p2)); //ERRO
    p3.exibir();

    return 0;
}
```

Template de funções e sobrecarga de operador

- É preciso indicar como comparar dois objetos Pessoa
 - A função maior() não sabe como fazer isso
- Solução
 - sobrecarga do operador de comparação >
 - friend permite conceder acesso especial a partes privadas de uma classe para funções ou outras classes específicas

```
class Pessoa {
private:
    string nome;
    int idade;

public:
    Pessoa(string n, int i): nome{n}, idade{i}{
    }

    void exibir(){
        cout << "Nome: " << nome << ", Idade: " << idade << endl;
    }

    friend bool operator> (const Pessoa& p1, const Pessoa& p2){
        return (p1.idade > p2.idade);
    }
};

template <typename T>
T maior(T x, T y) {
    return (x > y) ? x : y;
}

int main() {
    Pessoa p1("Maria", 25);
    Pessoa p2("Joao", 32);

    Pessoa p3(maior(p1, p2));
    p3.exibir();

    return 0;
}
```

Especialização do template de classe

- Classe Armazena
 - Permite guardar um dado de um determinado tipo
- E se o comportamento genérico não é adequado para um tipo específico?

```
template <typename T>
class Armazena {
private:
    T dado;
public:
    Armazena(T valor): dado{valor}{

    void exhibir(){
        cout << "Dado: " << dado << endl;
    }
};

int main() {
    Armazena<int> d1(15);
    Armazena<double> d2(25.5);
    Armazena<string> d3("Andre");

    d1.exibir();
    d2.exibir();
    d3.exibir();

    return 0;
}
```


Especialização do template de classe

- Podemos fornecer uma implementação diferente da classe template para um determinado tipo
 - Funciona como a sobrecarga de classe para aquele tipo
 - Indicação de `template<>` e a definição da classe com o tipo desejado

Especialização do template de classe

- Modifica o comportamento da classe/ funções membros
 - Função `exibir()` foi adaptada para exibir valores do tipo `double` em notação científica

```
template <typename T>
class Armazena {
private:
    T dado;
public:
    Armazena(T valor): dado{valor}{ }

    void exibir(){
        cout << "Dado: " << dado << endl;
    }
};

template <>
class Armazena<double> {
private:
    double dado;
public:
    Armazena(double valor): dado{valor}{ }

    void exibir(){
        cout << "Dado: " << scientific << dado << endl;
    }
};
```

Especialização do template de classe

- Apenas a função `exibir()` foi modificada
 - Não faz sentido fazer a especialização de toda a classe
- Podemos limitar a especialização a funções específicas
 - Colocar o identificador da classe e o operador de resolução de escopo antes da função
 - Menor quantidade de código repetido
 - Promove reutilização, a manutenibilidade e a eficiência do código

```
template <typename T>
class Armazena {
private:
    T dado;
public:
    Armazena(T valor): dado{valor}{}

    void exibir(){
        cout << "Dado: " << dado << endl;
    }
};

template <>
void Armazena<double>::exibir(){
    cout << "Dado: " << scientific << dado << endl;
}
```

Template para struct

- Templates também podem ser aplicados a structs
 - Permite a criação de estruturas de dados genéricas que funcionam com diferentes tipos
- struct e class são essencialmente equivalentes em C++
 - Com a diferença de que struct tem membros públicos por padrão

```
template <typename T>
struct Ponto2D {
    T x;
    T y;

    void exibir() const {
        cout << "(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Ponto2D<int> p1{3, 4};
    Ponto2D<double> p2{3.5, 7.8};
    Ponto2D<string> p3{"esquerda", "cima"};

    p1.exibir();
    p2.exibir();
    p3.exibir();

    return 0;
}
```

Template para struct

- Segue as mesmas regras do template de classe
 - Os membros da struct podem ter tipos pré-determinados
 - É instanciada uma única vez para cada tipo
 - Instanciação é explícita
 - Podemos definir múltiplos tipos em um mesmo template

```
template <typename T>
struct Ponto2D {
    T x;
    T y;

    void exibir() const {
        cout << "(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Ponto2D<int> p1{3, 4};
    Ponto2D<double> p2{3.5, 7.8};
    Ponto2D<string> p3{"esquerda", "cima"};

    p1.exibir();
    p2.exibir();
    p3.exibir();

    return 0;
}
```

Material Complementar

- Aula 33 - POO: Template de Função
 - <https://youtu.be/Ig2hWgn26jk>
- Aula 34 - POO: Template de Classe
 - https://youtu.be/9vwD_2o3QqQ